Project code: chat-client

The code for this example project is in the ch02/client-chat folder. The appendix at the end of this book offers instructions on how to set up and create a secret key for OpenAI and other providers.

Option 1: Using Google (default)

You can use that example using Google Gemini by running the following command in the source folder from the project root directory:

```
$ npm run start -w ch02/client-chat
```

You'll also need a Gemini API key to access their REST API:

```
// .env
GEMINI_API_KEY=<SECRET_KEY>
```

Option 2: Using OpenAI

You can use that example using OpenAI Generative by running the following command in the source folder:

```
$ npm run start:openai -w ch02/client-chat
```

You'll also need an OpenAI API key to access their REST API:

```
// .env
OPENAI_API_KEY=<SECRET_KEY
```

WARNING Make sure that your OpenAI API key is valid to avoid any problems during the installation process. Remember, this secret key should be kept confidential and not shared publicly.

2.2.4 Running the project

If you followed the setup instructions to run the chat-client example project, you should be able to visit http://localhost:5173 and interact with the agent. Figure 2.3 shows a screenshot of the application after some exchange of messages.

A QUICK CHAT WALK-THROUGH

Before we look at the implementation details, let's briefly walk through a simple chat exchange with Astra AI to see the application in action. When you first launch the application, you'll be presented with a welcoming screen, ready for your input. Locate the text box at the bottom of the screen and type your first prompt:

```
Tell me more about yourself
```